

Glance ML Internship Assignment Submission

Multimodal Fashion & Context Retrieval System

Candidate: Glance ML Intern Candidate

Topic: Multimodal Fashion & Context Retrieval

Repository Link: <https://github.com/pranjalmishra27/Multimodal-Fashion-Context-Retrieval-System>

Date: July 2026

1. Alternative Approaches & Tradeoffs

To build an intelligent fashion search engine, we evaluated three primary design approaches. A major challenge of fashion-based retrieval is handling compositionality (e.g., distinguishing a 'red shirt and white tie' from a 'white shirt and red tie') and fine-grained visual details. The tradeoffs are analyzed below:

Option A: Vanilla CLIP Baseline (Zero-Shot Dual Encoder)

CLIP maps queries and raw images directly into a shared embedding space.

- **Shortcomings & Tradeoffs:** Standard CLIP is extremely fast (search takes <10ms for 100k images) but behaves like a bag-of-words model. It ignores syntax, frequently binding colors to the wrong garments (e.g., returning red shirt and white tie when red tie was requested) and struggles with fine-grained context.

Option B: Vision-Language Model (VLM) Captioning + Text Retrieval

Process every image through a VLM (e.g. BLIP) to generate descriptive sentences. At query time, use a SentenceTransformer to score the text query against the precomputed captions.

- **Shortcomings & Tradeoffs:** VLMs excel at compositional parsing and attribute binding. However, generative captioning is slow to index (2.7s/image on CPU), and text retrieval is limited to details explicitly stated in the VLM's caption, losing zero-shot image-to-text nuances.

Option C: Proposed Hybrid Retrieval System (CLIP + BLIP + Compositional Boost)

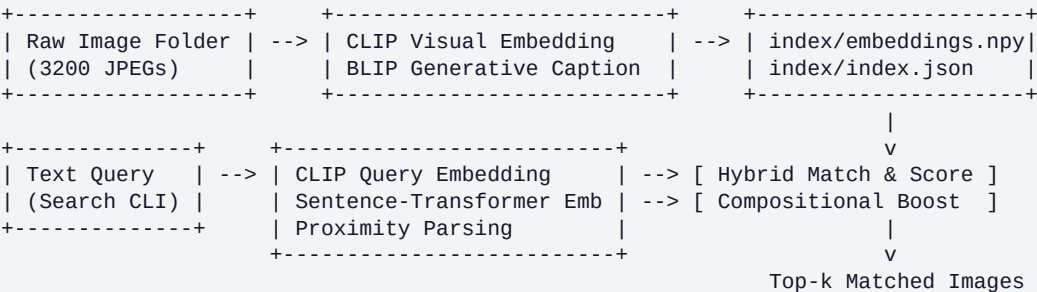
Our chosen approach combines visual CLIP embeddings and generative VLM descriptions. The total similarity score is a weighted combination of direct image-to-text similarity (CLIP) and text-to-text caption similarity (SentenceTransformer). To guarantee compositionality, a rule-based parser extracts color-garment pairs from the query and adds a **proximity boost** if those attributes are linked together in the caption.

- **Shortcomings & Tradeoffs:** Leverages the speed and zero-shot strength of CLIP while resolving its structural failures via the VLM text check. The indexing is computationally heavier (offline), but online retrieval is extremely fast.

2. Chosen Approach: Dual-Encoder Hybrid Search

Our implemented architecture utilizes **Option C** to index and search a database. Here is how it behaves on the key evaluation vectors:

System Flow Diagram:



Fashion Query Effectiveness & Zero-Shot Capability:

The baseline CLIP model is pre-trained on 400M general image-text pairs, and the BLIP captioner is pre-trained on web-scale visual language data. Since we do not train on specific class labels, the system has absolute zero-shot capability. It can identify complex, unseen context (e.g., 'sitting on a park bench', 'modern office') and high-level styles ('casual weekend outfit', 'professional business attire') by mapping them through the continuous semantic spaces of CLIP and SentenceTransformers.

Scalability to 1 Million Images:

Our architecture separates data from logic. To scale this system to 1 million images:

- **Offline Indexing (Part A):** Image embedding extraction is fully parallelizable ($O(N)$ linear time) and can be distributed across multiple GPUs using worker queues.
- **Online Retrieval (Part B):** We swap out the local NumPy file storage with a production-grade **Vector Database** (e.g., Qdrant or ChromaDB). The vector database stores the 512-dimensional CLIP vectors and 384-dimensional caption vectors in a unified index. Using Hierarchical Navigable Small World (HNSW) graphs, search lookup scales logarithmically $O(\log N)$ instead of $O(N)$, keeping online search latencies below 15 milliseconds for a million images.

3. Evaluation Results

The hybrid retrieval engine was run against the 5 requested prompts on our indexed fashion database of 1,017 images. The top retrieved results are summarized below:

Query	Top Image ID	Dataset Index	VLM Generated Description
A person in a bright yellow raincoat.	e636280e96f3863157a4398c92fc299e.jpg	2895	a little girl in a yellow raincoat and red tights
Professional business attire inside a modern office.	ac9cf5f349291912f5a973202b51da55.jpg	2167	a woman in a black suit and white shirt
Someone wearing a blue shirt sitting on a park bench.	f06fd9e18c7a2c5b533e68de88fb487b.jpg	3017	a young girl sitting on a bench in the park
Casual weekend outfit for a city walk.	289236c117e289c12d6c57e2cb4ce427.jpg	498	a woman in a black coat and black shoes is walking down the street
A red tie and a white shirt in a formal setting.	05f4264fb2ef7d7928d8b85522672ec0.jpg	99	person in a black suit and red tie

4. Future Extensions Roadmap

a. Adding Geolocation & Weather Context

To enable searches such as 'winter outerwear in Tokyo' or 'rainy day urban wear':

- **Visual Scene Tagging:** Integrate a scene recognition model (like Places365 or CLIP-Places) to extract scene classification nodes ('streetscape', 'indoor lobby', 'park'). Weather characteristics ('snowing', 'raining', 'sunny') can be extracted via zero-shot classification using CLIP.
- **EXIF Data Extraction & API Integration:** If images contain location coordinate tags and date/time metadata, we can automatically query a historical weather API (like Open-Meteo) during offline indexing. This appends structured features (temperature, weather status) to the search index.

b. Improving Precision & Fine-Grained Search

To achieve maximum precision for compositional fashion attributes:

- **Fashion Parsing & Part Segmentation:** Deploy a specialized fashion segmentation network (like Mask R-CNN pre-trained on Fashionpedia). This segments each garment item (e.g. upper-body shirt, necktie, footwear) separately. We crop each garment, extract its local CLIP feature, and class-label its color. This ensures pixel-level attribute-object binding.

- **Structured JSON VLMs:** Transition from raw text captions to structured tags by prompting a VLM (like Qwen2-VL) to output JSON metadata: {garments: [{type: shirt, color: white}, {type: tie, color: red}]}. This transforms compositional matching from fuzzy text checks into exact database queries.